University of Tehran
College of Engineering
School of Electrical and Computer Engineering

Distributed Systems

HW2: Distributed RPC, Services & Pub/Sub

Full Name

Sepanta Ghonoodi
Sajad Hasanpoor

Student ID

810102483
810102432

Spring 2026

0. Contents

1. Part 1: Familiarization with RPC and Technologies

1.1. Introduction to RPC

Remote Procedure Call (RPC) is a powerful inter-process communication (IPC) protocol that allows a computer program to cause a subroutine or procedure to execute in another address space (commonly on another computer on a shared network) without the programmer explicitly coding the details for this remote interaction.

1.2. The Core Idea of Remote Function Calls

The primary philosophy behind RPC is **transparency**. The goal is to make a remote network request look and feel exactly like a local function call. When a developer invokes a function, the underlying RPC framework abstracts away the complexities of network sockets, routing, HTTP protocols, and data packing, allowing developers to focus purely on the business logic rather than network infrastructure.

1.3. Local vs. Remote Call Differences

While RPC strives for transparency, fundamental differences exist between local and remote calls:

- **Memory Access:** Local calls share the same memory space (using pointers), whereas remote calls operate in isolated memory spaces, requiring data to be passed entirely by value over the network.
- **Latency:** A remote call involves network transmission, making it orders of magnitude slower than a local CPU instruction.
- **Failure Modes:** Local calls typically only fail due to logical application errors or hardware crashes. Remote calls introduce partial failures, such as network timeouts, packet loss, or server unavailability.

1.4. The Role of Client Stub and Server Stub

The illusion of a local call is maintained using ”Stubs”:

- **Client Stub:** Acts as a proxy on the client side. It intercepts the local function call, marshals (packs) the parameters into a network message, and handles the OS-level network transmission.
- **Server Stub (Skeleton):** Receives the network message on the remote machine, unmarshals (unpacks) the parameters, and invokes the actual local function on the server. Once the function returns, it marshals the result back to the client.

1.5. Serialization and Deserialization

Data structures in memory (like Go structs or Python objects) cannot be transmitted directly over a network cable. **Serialization** (Marshalling) is the process of translating these in-memory data structures into a standardized byte stream (e.g., JSON, XML, or Protocol Buffers). **Deserialization** (Unmarshalling) is the reverse process, reconstructing the byte stream back into native objects on the receiving end.

1.6. Interface Definition Language (IDL)

An IDL is a standardized, language-agnostic way to describe the interfaces, methods, and data types of an RPC service. By using an IDL (such as `.proto` files in gRPC), developers can define the service contract once and automatically generate the necessary Client and Server Stubs in multiple programming languages (e.g., Go, Python, Java).

1.7. Common Errors in RPC

Distributed systems are inherently unreliable. Common RPC errors include:

- **Timeouts:** The server takes too long to process, or the network drops the packet.
- **Network Partitions:** The connection between the client and server is physically or logically severed.
- **Serialization Mismatch:** The client and server are using incompatible versions of the IDL schema.
- **Idempotency Issues:** Retrying a failed RPC call might cause unintended duplicate executions if the original request was actually processed but the response was lost.

1.8. Comparing RPC with REST

While both are used for building distributed APIs, their paradigms differ:

- **RPC is Action-Oriented:** It focuses on behaviors and procedures (e.g., `CalculateTax()`, `AuthenticateUser()`). It typically uses a single endpoint and sends the function name within the payload.
- **REST is Resource-Oriented:** It focuses on entities (e.g., `/users/123`) and uses standard HTTP verbs (GET, POST, PUT, DELETE) to manipulate the state of those resources.

1.9. Technology Comparison

The following table compares three major RPC technologies: gRPC (which we utilized in our project), JSON-RPC, and XML-RPC.

Technology	Data Format	IDL Support	Streaming	Primary Use Case
gRPC	Protocol Buffers (Binary)	Yes (`.proto`)	Yes (Bi-directional)	High-performance microservices, Polyglot systems.
JSON-RPC	JSON (Text)	Optional / None	No	Simple, lightweight web APIs, Blockchain interfaces.
XML-RPC	XML (Text)	No	No	Legacy systems, WordPress APIs.

Table 1: Comparison of common RPC Technologies

1.10. Analytical Questions

Q1: Why does RPC make calling a remote service feel like calling a local function?
Because the RPC framework automatically generates Client and Server Stubs. The programmer simply calls a method on the local Stub object. The Stub abstracts away the complexities of serialization, socket programming, and HTTP headers, making the API surface identical to a local method signature.

Q2: Why can this similarity be dangerous or misleading?
Because it hides the realities of the network. A developer might mistakenly invoke an RPC call inside a tight `for`-loop, assuming it executes in nanoseconds like a local function, while in reality, each call incurs milliseconds of network latency, severely bottlenecking the system.

Q3: What errors can occur in a remote call that do not exist in a local call?
Local calls do not suffer from network partitions, DNS resolution failures, packet drops, or TLS handshake errors. Furthermore, in RPC, you can encounter the ”Two Generals’ Problem”—if a client receives

a timeout, it is impossible to know whether the server crashed before processing the request, or if it processed the request successfully but the response was lost in transit.

Q4: Under what conditions is gRPC a better choice than REST?
gRPC excels in internal microservice-to-microservice communication where performance is critical. Because it uses binary Protocol Buffers and HTTP/2, it has drastically lower payload sizes and CPU overhead compared to JSON over HTTP/1.1. It is also ideal when strict type-safety across different programming languages is required, or when real-time streaming (like our file download implementation) is necessary.

Q5: Under what conditions is REST simpler or more appropriate?
REST is the superior choice for public-facing web APIs intended to be consumed directly by web browsers or external third-party clients. Browsers have native support for HTTP verbs and JSON, making REST inherently easier to debug with standard browser dev-tools. Additionally, REST's stateless, cacheable nature makes it highly scalable for read-heavy resources (like fetching articles or user profiles).

2. Part 2: Multi-VM Distributed System Implementation

2.1. System Architecture Overview

Our distributed system consists of three independent Virtual Machines (VMs), designed to strictly decouple user-facing HTTP traffic from backend security and storage operations.

- **VM1 (Web Server):** The user-facing gateway. It serves HTML templates, captures user inputs, and routes requests to the appropriate backend over the network. It purposefully does not store any local states, files, or passwords.
- **VM2 (Auth Server):** The centralized identity provider. It securely holds the `users.json` database and evaluates authentication attempts.
- **VM3 (File Server):** The secure file repository. It provides chunk-based file downloads exclusively to authenticated sessions.

2.2. Network Configuration and Isolation

To ensure true distributed communication, all services were deployed on separate instances. The `ifconfig` outputs confirm their distinct IP allocations within the network, proving that IPC occurs over real network sockets rather than `localhost`.

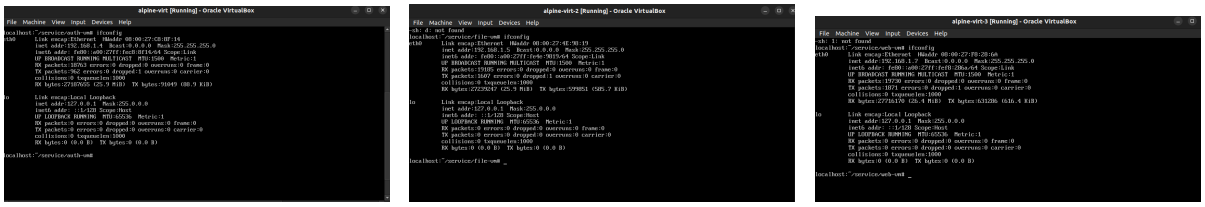

Figure 1: Network Configurations: VM1 (left), VM2 (center), and VM3 (right) confirming physical network separation.

2.3. Service Initialization and Startup

Before accepting requests, the infrastructure must be correctly bootstrapped. VM2 loads the user database into memory, while VM3 mounts the file system. Subsequently, the gRPC listeners are bound to their respective network ports.

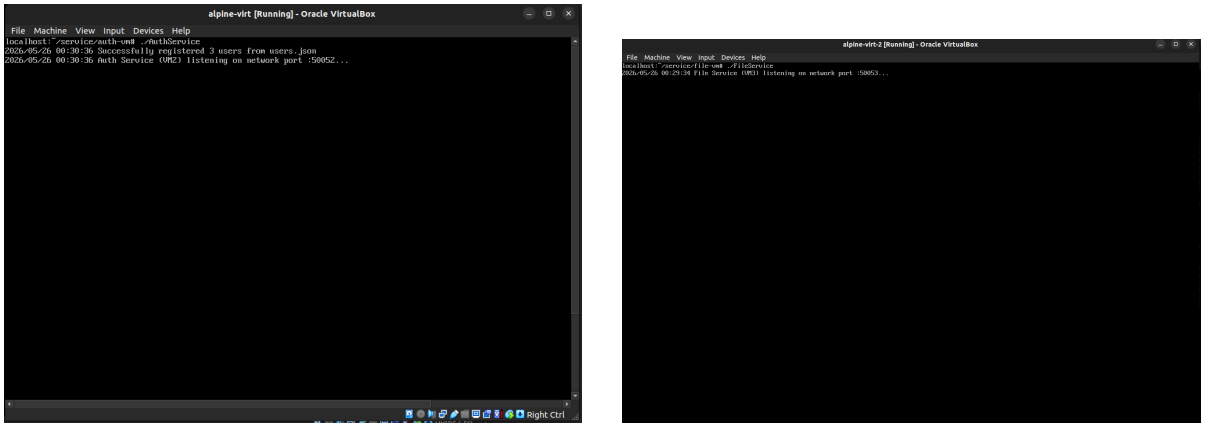

Figure 2: Database and file system initialization during the service bootstrap phase.

Figure 3: Active console sessions running VM1 (Web), VM2 (Auth), and VM3 (File).

2.4. RPC Technology Selection: gRPC

We elected to use **gRPC** powered by Protocol Buffers for our inter-process communication instead of a standard HTTP REST API. The primary reasons for this architectural decision include:

- **Strict Typing:** Protobuf guarantees strong typing for payloads, eliminating runtime casting errors common in JSON-based APIs.
- **High Performance:** gRPC utilizes HTTP/2, allowing for multiplexed streams and binary serialization, which drastically reduces network overhead.
- **Native Streaming Support:** Unlike traditional RPCs, gRPC natively supports server-to-client streaming, which is essential for our chunked file transfer in VM3.

2.5. Authentication Procedure and Security

The authentication flow begins on VM1 via an HTML login form. To ensure security, the dashboard route is strictly protected against unauthenticated access.

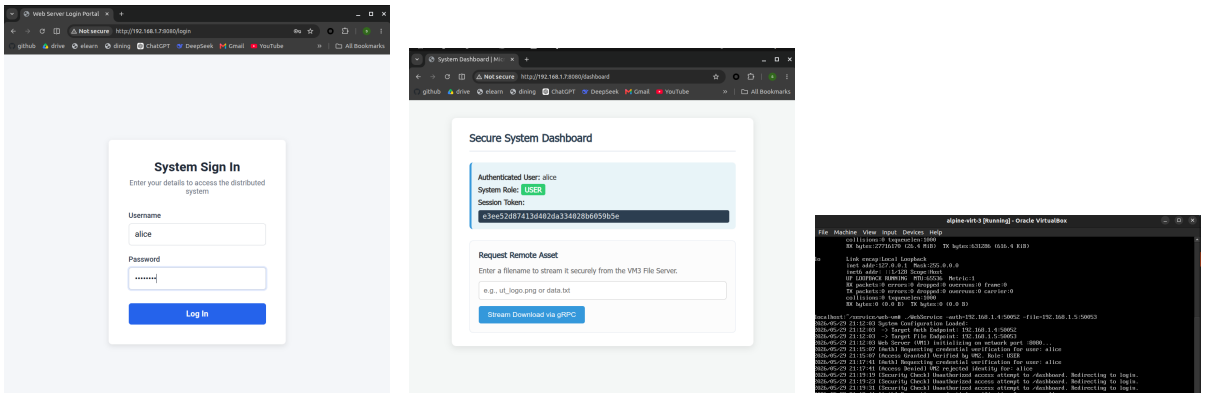

Figure 4: The Sign-In/Login interfaces (left, center) and the system blocking an unauthorized direct access attempt to the dashboard (right).

Upon form submission, VM1 initiates a gRPC call to VM2 invoking the `Login` procedure. A critical design enhancement in our system is the implementation of the `bcrypt` hashing algorithm. Passwords are not compared in plain text; instead, VM2 utilizes `bcrypt.CompareHashAndPassword` to safely validate credentials.

2.5.1 Scenario A: Failed Authentication

When invalid credentials are provided, VM1 forwards the request via gRPC to VM2. VM2 rejects the request, and VM1 displays a localized error without exposing internal service states.

Figure 5: Failed Flow: UI error message (left), VM1 forwarding the request (center), and VM2 rejecting the invalid password (right).

2.5.2 Scenario B: Successful Authentication

Upon entering valid credentials, VM2 verifies the bcrypt hash and generates a secure 16-byte hex token. This token is returned to VM1, which caches it as an `HttpOnly` session cookie and redirects the user to the dashboard.

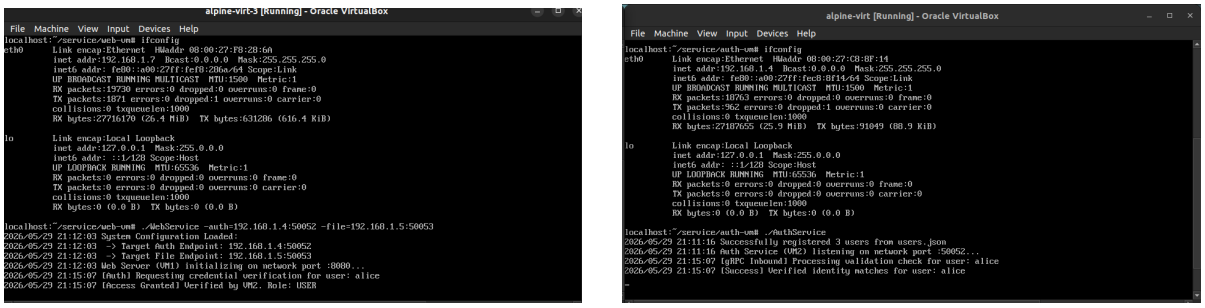

Figure 6: Successful Flow: VM1 receiving valid inputs (left) and VM2 verifying the identity and granting access (right).

2.6. Secure File Transfer Mechanism

To retrieve files from VM3, VM1 establishes a gRPC connection and attaches the user's session token into the gRPC **metadata** context (acting as a secure authorization header).

The `DownloadFile` procedure in VM3 intercepts this context, validates the token, and opens the requested file. Because reading massive files entirely into RAM could cause out-of-memory crashes, we implemented a robust Server-Streaming architecture. The file is read in **4096-byte chunks** and continuously piped over the gRPC stream (`stream.Send`) to VM1, which simultaneously flushes the bytes to the client's HTTP response writer.

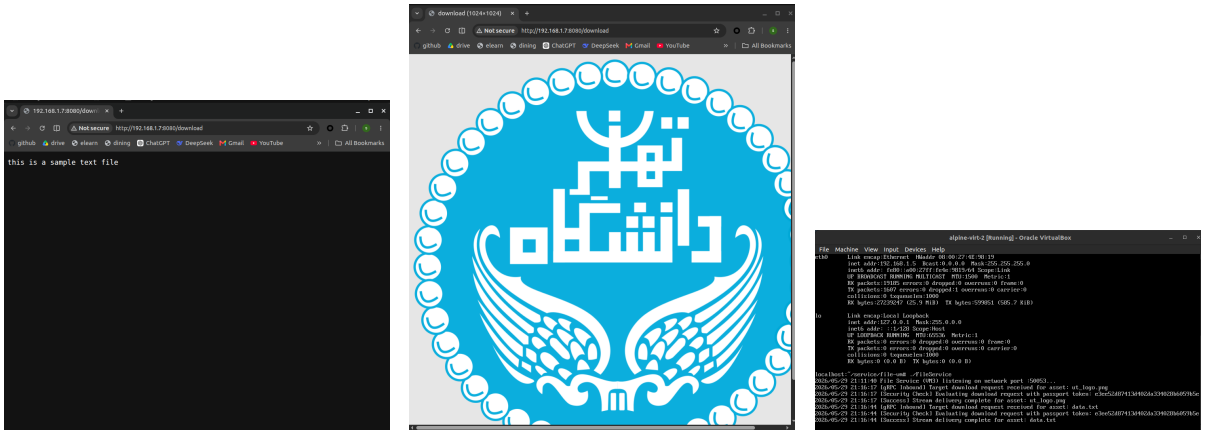

Figure 7: Secure File Transfer: Downloading a text file (left), an image file (center), and the corresponding chunked streaming logs on VM3 (right).

3. Part 3: Pub/Sub Architecture and Memory Monitoring

3.1. Memory Monitoring Strategy

To maintain system observability, we integrated a background daemon (goroutine) inside VM1's web server. This monitor wakes up every 5 seconds and polls the Go runtime memory statistics using the native `runtime.ReadMemStats(&m)` function. Specifically, we track the `Alloc` metric, which represents the bytes currently allocated for active heap objects.

3.2. The Pub/Sub Mechanism

We implemented a lightweight, HTTP-based Publish/Subscribe (Pub/Sub) pattern to manage memory alerts.

- **Publisher (VM1):** If the heap allocation exceeds the predefined threshold of 300 MB, VM1 constructs a structured `MemoryEvent` JSON payload containing the exact memory usage, threshold, and timestamp. It then executes an HTTP POST request to the Subscriber's endpoint.
- **Subscriber (Port 9090):** A standalone Go process running an HTTP listener at the `/alert` endpoint. It acts as the event sink, listening for incoming POST requests, deserializing the JSON payload, and triggering a critical console alert.

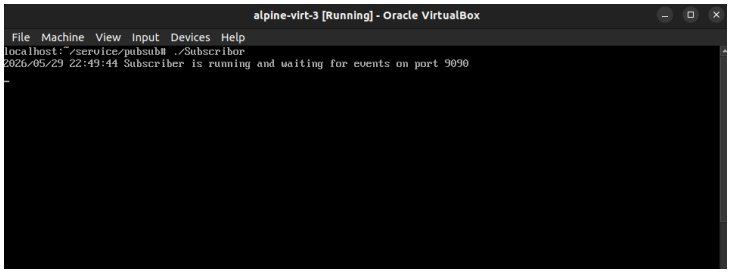

Figure 8: Initialization of the Subscriber process, actively listening on port 9090 for incoming memory alerts.

3.3. Memory Inflation and Testing Strategy

To deliberately trigger the alarm, we exposed a specialized `/consume-memory` endpoint on VM1. When invoked (e.g., via `curl ?mb=100`), the server allocates a massive byte slice and appends it to a global 2D array (`leakedMemory`). By attaching the chunks to a global variable, we prevent the Go Garbage Collector (GC) from immediately freeing the memory, successfully simulating a severe memory leak.

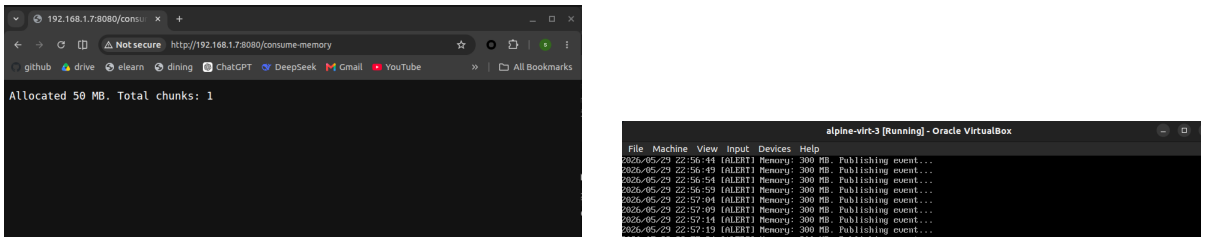

Figure 9: Triggering the memory leak via the `/consume-memory` endpoint (left) and the subsequent threshold breach detected by the VM1 monitor (right).

3.4. Event Publishing and Reception

Once the memory threshold is breached, the Publisher formulates the JSON event and dispatches it over the network. The Subscriber immediately intercepts this event, parses the critical payload, and outputs a formatted alert to the system administrator.

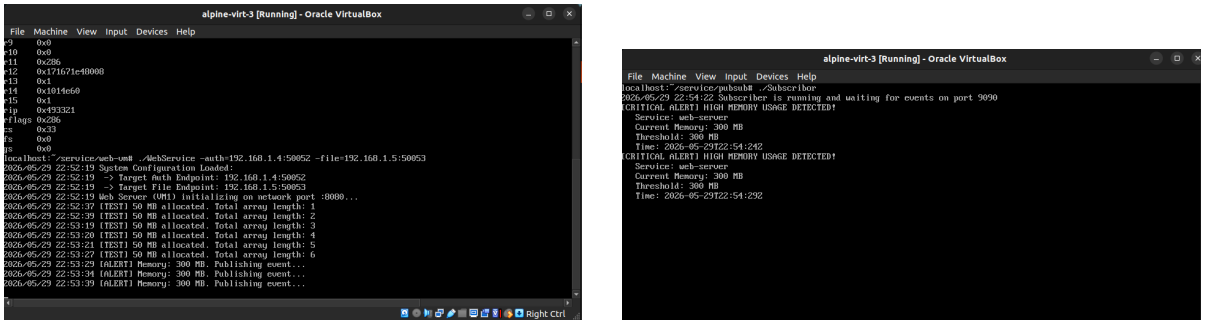

Figure 10: The Publisher dispatching the HTTP POST event (left) and the Subscriber actively receiving the JSON payload (right).

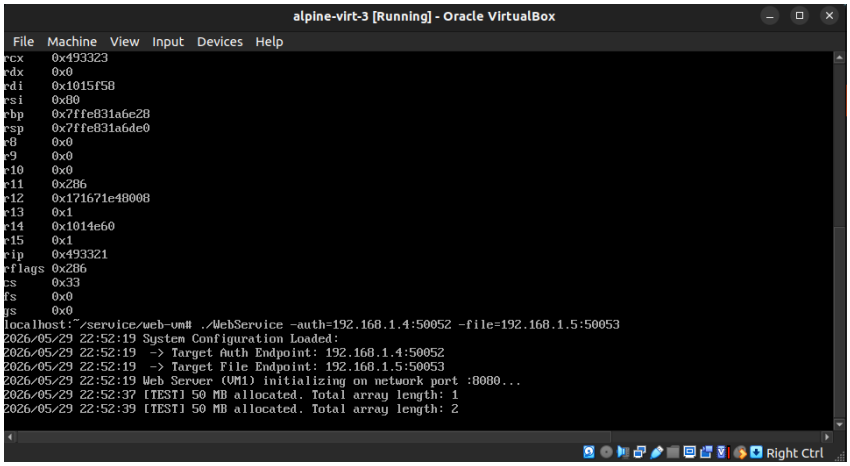

Figure 11: The final output on the Subscriber console, clearly displaying the critical memory alert and relevant system metadata.

3.5. Implementation Challenges and Limitations

During development, we encountered a few architectural challenges:

- **Garbage Collection Interference:** Initially, our memory leak simulation failed because the allocated arrays fell out of scope and were immediately swept by Go's aggressive Garbage Collector before the 5-second monitor cycle could detect the spike. We resolved this by persisting the allocations in a global slice.
- **Stream Synchronization:** Handling `io.EOF` during the gRPC file stream required careful synchronization to ensure the HTTP response writer did not abruptly terminate before the final data chunk was successfully delivered to the browser.